PYTHON PROGRAMMING — DIGITAL NOTES

Comprehensive Master Reference Guide • Syntax, Data Structures, OOP, Best Practices & Core Concepts

1. Language Fundamentals & Data Types

Primitive Data Types

Python is dynamically and strongly typed. Everything in Python is an object.

Type	Example	Mutability
`int`	`42, -5`	Immutable
`float`	`3.14159`	Immutable
`str`	`"Hello"`	Immutable
`bool`	`True / False`	Immutable
`NoneType`	`None`	Immutable

String Manipulation

```
# Slicing: [start:stop:step]
s = "Python Programming"
s[0:6]      # 'Python'
s[::-1]     # 'gnimmargorP nohtyP' (Reverse)

# String Formatting (f-strings)
name, score = "Alice", 95.456
msg = f"{name} scored {score:.2f}"

# Key Methods
s.strip(), s.split(','), "-".join(['a','b'])
```

Variables & Scope Rule (LEGB)

Python resolves names using the **LEGB** order:

- **L**ocal — Inside current function
- **E**nclosing — Outer enclosing functions
- **G**lobal — Top-level module
- **B**uilt-in — Preloaded keywords/built-ins

```
x = "global"
def outer():
    x = "enclosing"
    def inner():
        nonlocal x  # Modifies enclosing 'x'
        x = "inner modified"
    inner()
```

Type Casting & Operators

```
# Operations & Division
7 / 2   # 3.5 (Float Division)
7 // 2  # 3   (Floor/Integer Division)
7 % 2   # 1   (Modulus / Remainder)
2 ** 3  # 8   (Exponentiation)

# Identity vs Equality
a == b  # Compares VALUES
a is b  # Compares MEMORY ADDRESS (id)
```

2. Core Data Structures

Lists & Tuples

- **List**: Ordered, mutable sequence Mutable
- **Tuple**: Ordered, immutable sequence Immutable

```
# List Operations (O(1) append/pop, O(n) insert/remove)
lst = [1, 2, 3]
lst.append(4)        # [1, 2, 3, 4]
lst.extend([5, 6])    # Appends items individually
lst.pop()            # Removes & returns last element

# Tuples & Unpacking
```

Dictionaries & Sets

- **Dict**: Key-Value pairs, Hash Map, O(1) avg look
- **Set**: Unordered collection of unique elements.

```
# Dictionary Operations
d = {"name": "Alice", "age": 25}
val = d.get("city", "N/A")  # Safe acces

# Set Operations (Union, Intersection, D
s1 = {1, 2, 3}
s2 = {3, 4, 5}
```

```
point = (10, 20, 30)
x, y, z = point
a, *rest = [1, 2, 3, 4]  # a=1, rest=[2,3,4]
```

```
inter = s1 & s2  # {3}
union = s1 | s2  # {1, 2, 3, 4, 5}
```

Comprehensions (List, Set, Dict)

```
# List Comprehension: [expression for item in iterable if condition]
squares = [x**2 for x in range(10) if x % 2 == 0]  # [0, 4, 16, 36, 64]

# Dict Comprehension
char_count = {char: len(char) for char in ["apple", "banana"]}  # {'apple': 5, 'banana': 6}

# Generator Expression (Memory Efficient Lazy Evaluation)
gen = (x**2 for x in range(1000000))  # Yields items on demand
```

3. Functions & Advanced Parameters

Args, Kwargs & Lambda

```
# *args (tuple), **kwargs (dictionary)
def flex_func(*args, **kwargs):
    print(args)    # (1, 2)
    print(kwargs)  # {'a': 3, 'b': 4}

flex_func(1, 2, a=3, b=4)

# Anonymous Lambda Functions
add = lambda x, y: x + y
pairs = [(1, 'one'), (3, 'three'), (2, 'two')]
pairs.sort(key=lambda p: p[0])
```

Decorators

Wraps a function to modify or extend its behavior.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before execution")
        res = func(*args, **kwargs)
        print("After execution")
        return res
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")
```

4. Object-Oriented Programming (OOP)

Classes, Inheritance & Magic Methods

```
class Animal:
    def __init__(self, name: str):
        self.name = name          # Public attribute
        self._protected = "soft"   # Convention: Protected
        self.__private = "secret" # Name mangled: Private

    def speak(self):
        raise NotImplementedError("Subclasses must implement speak()")

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"
```

```
    def __str__(self):  # Human readable string representation
        return f"Dog({self.name})"

class MathUtils:
    @staticmethod
    def add(a, b): return a + b

    @classmethod
    def info(cls): return f"Utility class {cls.__name__}"
```

5. Exception Handling & File I/O

Try - Except - Else - Finally

```
try:
    num = int(input("Enter number: "))
    res = 10 / num
except ZeroDivisionError as e:
    print("Cannot divide by zero!")
except ValueError:
    print("Invalid integer input")
else:
    print("Executed if NO exception occurred")
finally:
    print("Always executes (cleanup)")
```

Context Managers (File Handling)

The `with` statement ensures resources are properly closed.

```
# Reading & Writing Files Safely
with open("data.txt", "w", encoding="utf-8") as f
    f.write("Hello World
")

with open("data.txt", "r") as f:
    content = f.read()  # Or f.readlines()
```

6. Essential Standard Library & Best Practices

Built-in Utility Modules

- `collections`: `Counter`, `defaultdict`, `namedtuple`, `deque`
- `itertools`: `permutations`, `combinations`, `chain`
- `json`: `json.dumps()` (serialize), `json.loads()` (parse)
- `os / sys`: OS file paths, CLI arguments (`sys.argv`)
- `datetime`: `datetime.now()`, formatting dates

PEP 8 & Common Pitfalls

> ⚠️ **Mutable Default Arguments Trap**
> Do NOT use mutable defaults like `def f(l=[])`! Lists persist across calls. Use `def f(l=None)` instead.

- **Shallow vs Deep Copy**: Use `copy.deepcopy()` for nested structures.
- **Naming Conventions**: `snake_case` for functions/vars, `PascalCase` for classes, `UPPER_CASE` for constants.